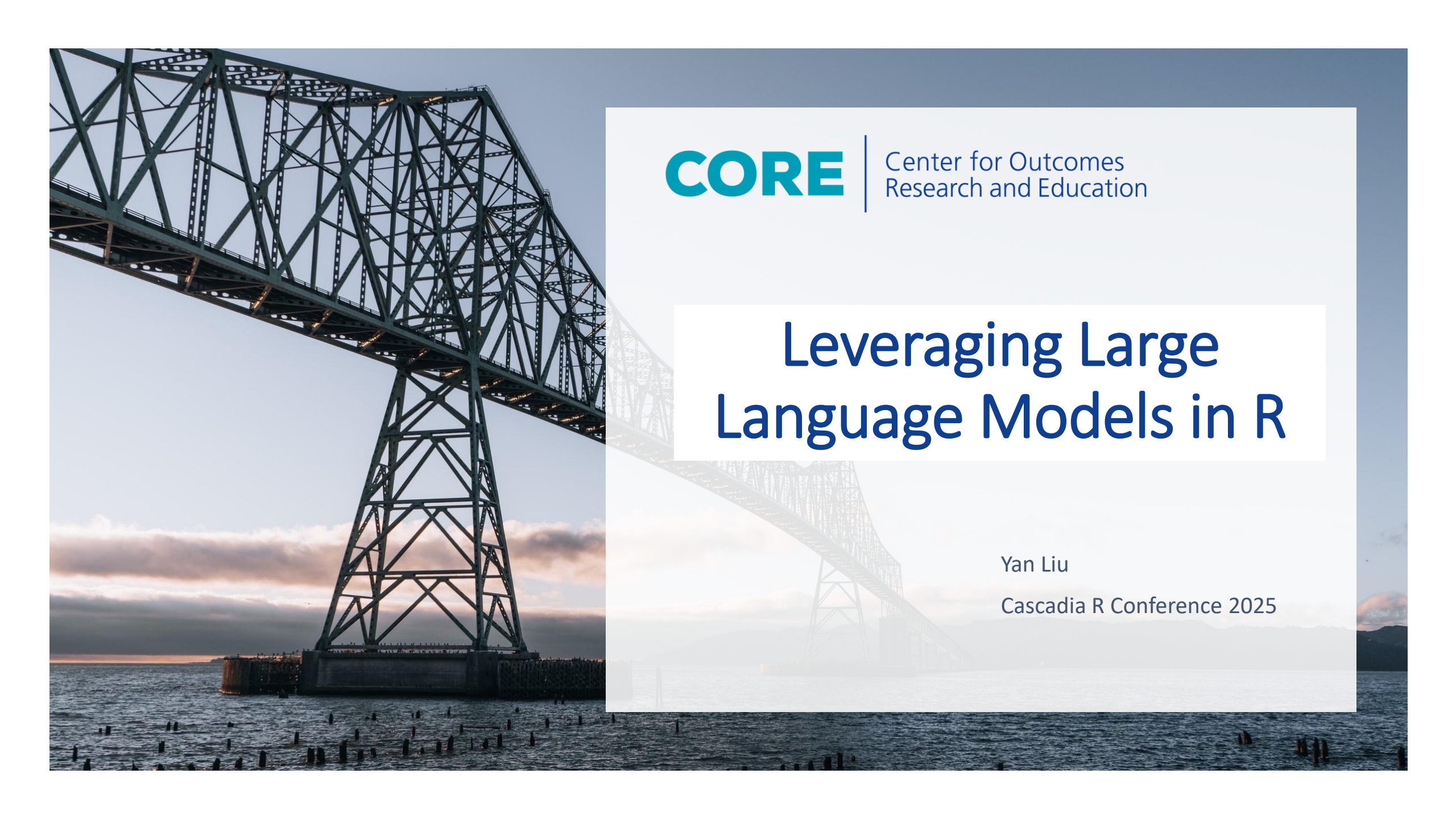




CORE

Center for Outcomes
Research and Education

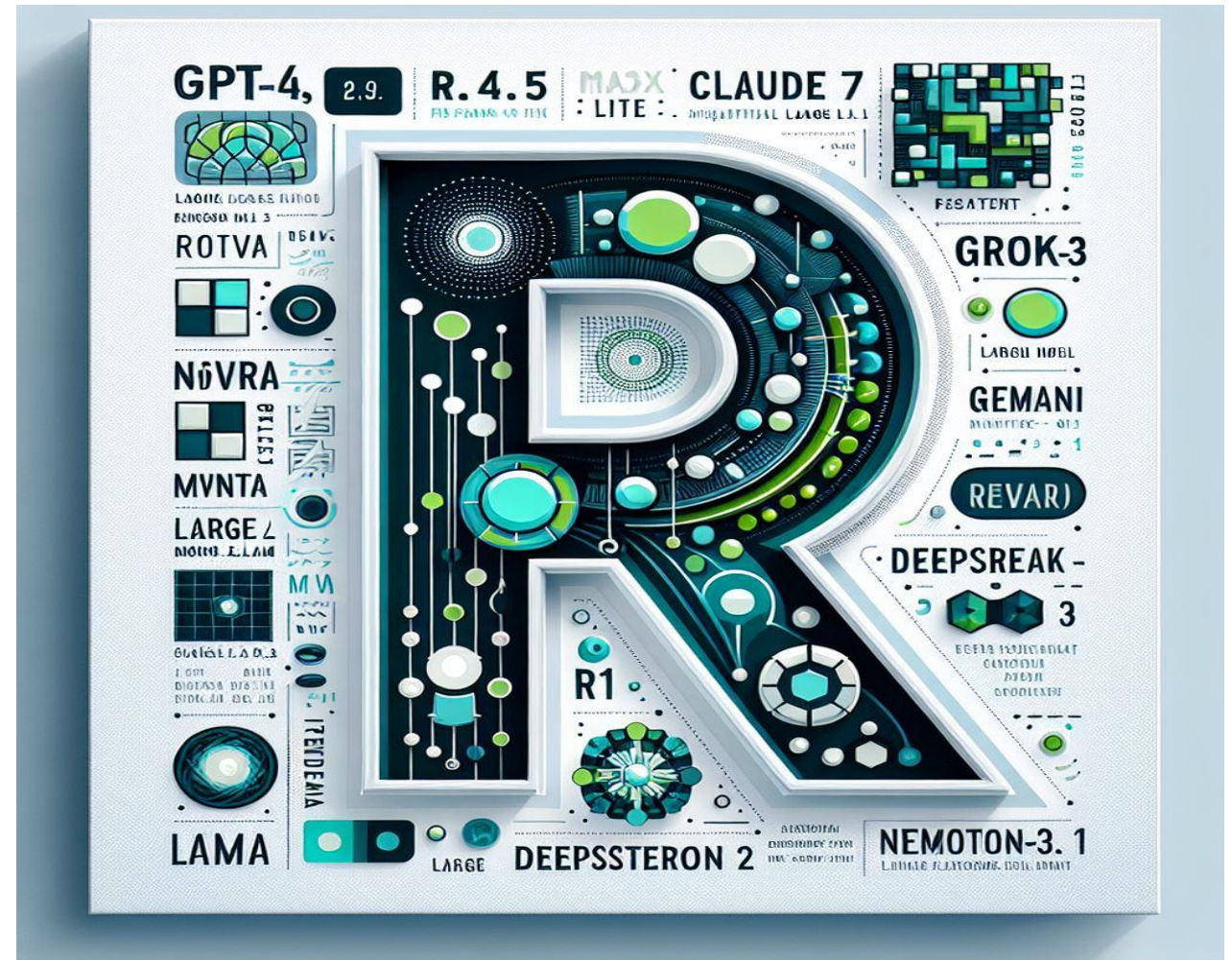
Leveraging Large Language Models in R

Yan Liu

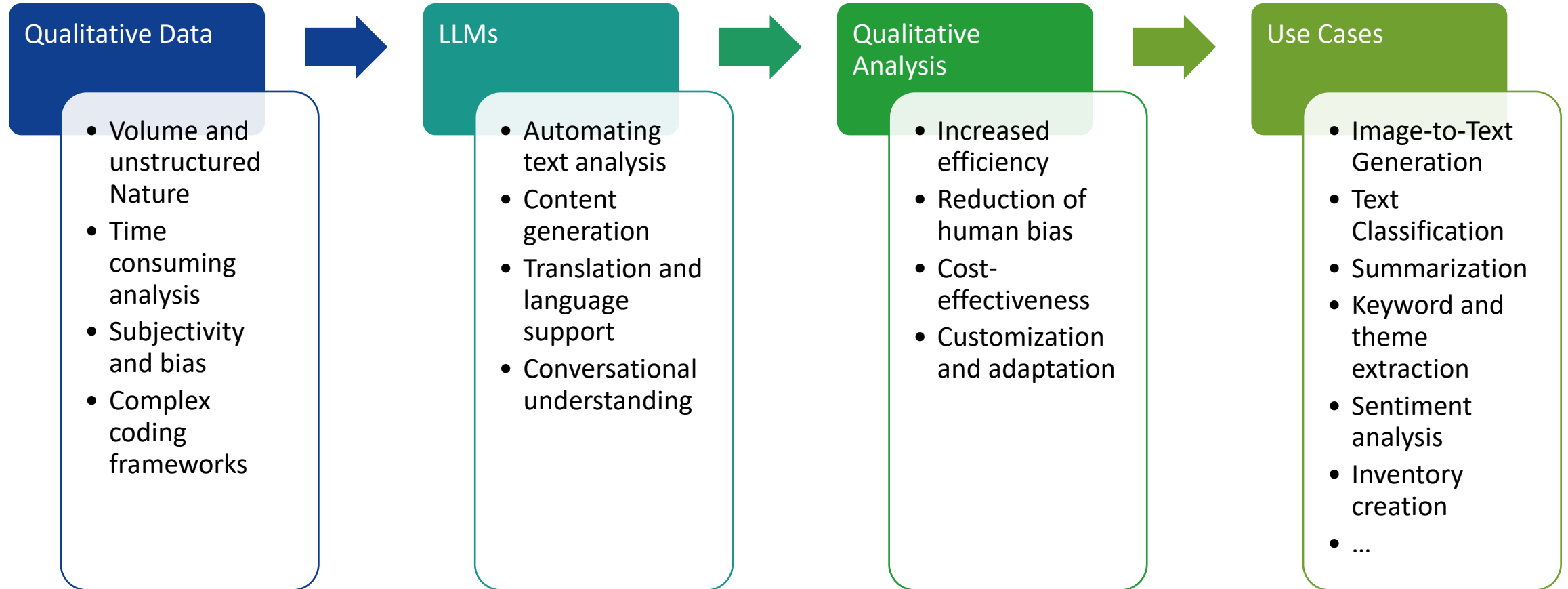
Cascadia R Conference 2025

Outlines

1. Introduction
2. Quick Overview Large Language Models and {ellmer}
3. Practical applications
4. Thoughts and Takeaways



The Challenge of Qualitative Data & support from LLMs



Source: Brown (2025). Quality statistics with large language models. Unpublished manuscript. Hayes, A. S. (2025). "Conversing" With Qualitative Data: Enhancing Qualitative Research Through Large Language Models (LLMs). International Journal of Qualitative Methods, 24. <https://doi.org/10.1177/16094069251322346> (Original work published 2025)

Overview of LLMs

How to start?

- Get API Key or download locally
- Call LLMs and work on your environment (CMD, R, python, etc.)

What LLMs are available for CORE to use?

- OpenAI API with Azure endpoint
 - GPT-4o
 - Gpt-4o mini
 - text-embedding-3-large
 - text-embedding-ada-002
- Open Source
 - Llama 3.2
 - Llama 3.3

CORE

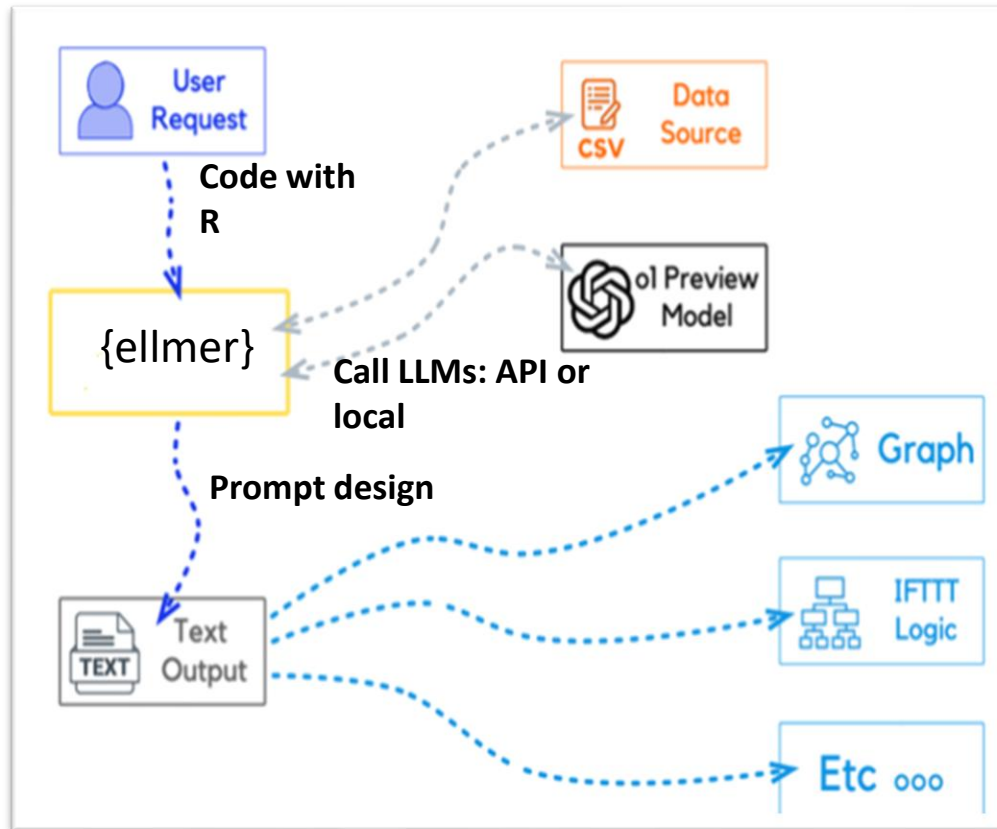
Best LLMs in 2025 (Update May 2025)		
Developer	LLM Name	Access
Open AI	GPT-4.5, GPT-4o, GPT-4o mini, GPT-4.1, GPT-3o, GPT-3o mini, GPT-1o, Sora	API
Anthropic	Claude 3.7 Sonnet, Claude 3.5 Sonnet	API
xAI	Grok-3, Grok-2	API
Google DeepMind	Gemini 2.5 Pro, Gemini 2.0 Flash-Lite, Gemini 2.0 Flash	API
Alibaba	Qwen 3, Qwen 2.5-Max	API, Open Source
DeepSeek	DeepSeek R1, DeepSeek-V3, DeepSeek-V2	API, Open Source
Amazon	Nova	API
Mistral AI	Mistral Large 2, Mixtral 8x22B	API
Meta AI	Llama 3.1	Open Source
Nvidia	Nemotron-4	Open Source
Microsoft	Phi-3	API, Open Source

Evidence for Change

Source: <https://explodingtopics.com/blog/list-of-llms>

Implement LLMs with R {ellmer}

- {ellmer} enables communication with LLMs from various providers.



- Anthropic's Claude: `chat_anthropic()`.
- AWS Bedrock: `chat_aws_bedrock()`.
- Azure OpenAI: `chat_azure_openai()`.
- Cloudflare: `chat_cloudflare()`.
- Databricks: `chat_databricks()`.
- DeepSeek: `chat_deepseek()`.
- GitHub model marketplace: `chat_github()`.
- Google Gemini/Vertex AI: `chat_google_gemini()`, `chat_google_vertex()`.
- Groq: `chat_groq()`.
- Hugging Face: `chat_huggingface()`.
- Mistral: `chat_mistral()`.
- Ollama: `chat_ollama()`.
- OpenAI: `chat_openai()`.
- OpenRouter: `chat_openrouter()`.
- perplexity.ai: `chat_perplexity()`.
- Snowflake Cortex: `chat_snowflake()` and `chat_cortex_analyst()`.
- VLLM: `chat_vllm()`.

Examples of {ellmer}

Chat_azure_openai()

```
```{r}
chat <- chat_azure_openai(
 endpoint = endpoint,
 deployment_id = "gpt-4o",
 api_version = "2024-06-01",
 system_prompt = "You are a sensor data analyst and R programmer",
 api_key = subscription_id,
 credentials = list('Authorization' = paste('Bearer', access_token),
 'Content-Type' = 'application/json')
)

chat$chat("Tell me what is t test in one sentence")
```
```

System prompt

User prompt

A t-test is a statistical method used to determine if there is a significant difference between the means of two groups, which may be related in certain features.

Chat_ollama()

Make sure you have download Ollama and the LLMs

```
```{r}
install ollama and pull the model
chat <- chat_ollama(model = "llama3.2",
 system_prompt = "You are a sensor data analyst
and R programmer")

chat$chat("Tell me a joke about R programmer in one sentence")
```
```

why did the R programmer cross the road, because it was trying to reach its "seam"-less convergence with its dataset?

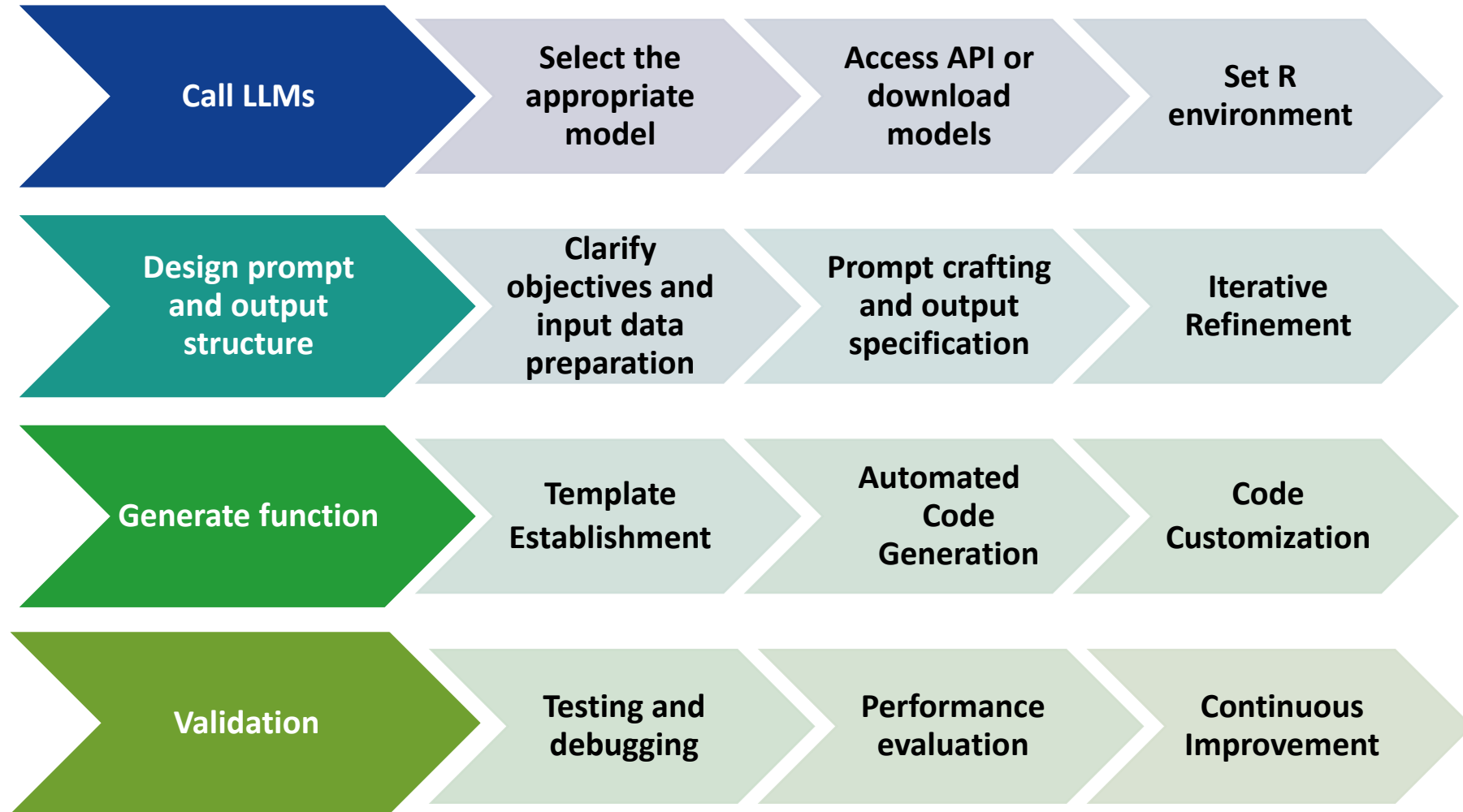
More {ellmer} features:

- Image input
- Structured data extraction
- Stream responses as they are received
- tool/function invocation
- Compare results from different models...

Practical applications

CORE

Workflow for use cases



Application 1: Image-to-Text Generation

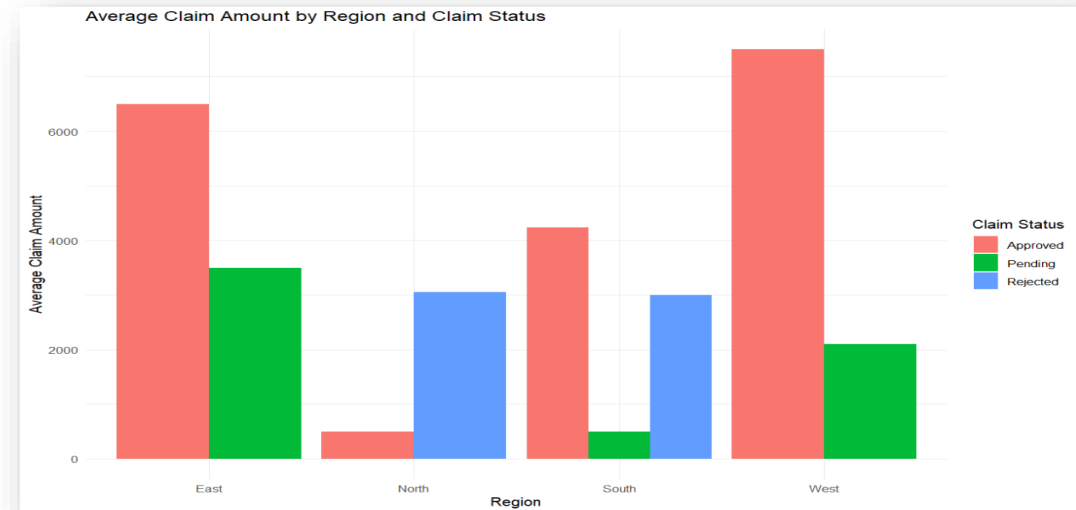
- **Problems:** Extracting structured text from images for research or automation
- **Example Solution:** Using {ellmer} provide a written description of a ggplot
- **Key Considerations:** Accuracy of the summary, prompt design

Prompt Design

```
chat <- chat_azure_core(  
  system_prompt = "You are a senior data analyst"  
)  
  
type_summary <- type_object(  
  "Summary of the plot.",  
  title = type_string("read the title of the plot. If not, give a title for the  
plot"),  
  x_axis = type_string("describe the x axis with detail, e.g value, type, range"),  
  y_axis = type_string("describe the y axis with detail,e.g value, type, range"),  
  legend = type_string("describe the legend in the plot"),  
  summary = type_string(summary_prompt)  
)  
  
data <- chat$extract_data(plot, type = type_summary)
```

CORE

Input Image



Output Text

```
$title  
[1] "Average Claim Amount by Region and Claim Status"  
  
$x_axis  
[1] "The x-axis represents different geographic regions, namely East, North,  
South, and West."  
  
$y_axis  
[1] "The y-axis represents the average claim amount in monetary units, ranging  
from 0 to 10,000."  
  
$legend  
[1] "The legend indicates three different claim statuses: Approved (red),  
Pending (blue), and Rejected (green)."  
  
$summary  
[1] "This bar chart illustrates the average claim amounts across different  
regions segmented by claim status categories. In the East and West regions, the  
approved claim amounts are notably higher (over 8000) compared to other  
statuses. The North region shows a higher pending claim amount compared to  
approved and rejected statuses, while the South has balanced pending and  
rejected amounts, both being relatively low compared to other regions. Rejected  
claims are generally lower in all regions except in the East and West, where  
they are closer to approved amounts."
```

Evidence for Change

Application 2: Text Classification

- **Problems:** Automating categorization of qualitative responses
- **Example Solution:** Using {ellmer} to classify text based on themes
- **Key Considerations:** Model reliability, handling ambiguous responses

Prompt Design

```
type_summary <- type_object(  
  all = type_string("Predict the sentiment category based on the content.  
  
Provide class probabilities for each of the following categories: Anxiety, Bipolar, Depression,  
Normal, Personality Disorder, Stress, and Suicidal.  
  
The probabilities should represent the likelihood of the text belonging to each category, summing  
up to 1.00.  
  
Use the following format for output:  
  
Anxiety: [Probability Value]  
Bipolar: [Probability Value]  
Depression: [Probability Value]  
Normal: [Probability Value]  
Personality Disorder: [Probability Value]  
Stress: [Probability Value]  
Suicidal: [Probability Value]  
  
Example for final output:  
  
Anxiety: 0.0111  
Bipolar: 0.0693  
Depression: 0.1111  
Normal: 0.7580  
Personality Disorder: 0.0073  
Stress: 0.0021  
Suicidal: 0.0412"))
```

Input data

Source: [Kaggle - Sentiment Analysis for Mental Health](#), data is sourced from diverse platforms including social media posts, Reddit posts, Twitter posts, and more.
Size: 53,043 labeled mental health text entries.
Each text has been classified by: Anxiety, Bipolar, Depression, Normal, Personality disorder, Stress, Suicidal.

Output results

| Model | Accuracy (compared to dataset classifications) |
|-------------------------|--|
| Logistic Regression | 71.00% |
| RoBERTa (Deep Learning) | 75.33% |
| GPT-4o | 76.95% |

Note: 20 mins for 1000 texts classification.



Application 3: Summarization

- Problems:** Condensing lengthy documents into key insights
- Example Solution:** {ellmer} prompt for summarizing reports/articles
- Key Considerations:** Balancing brevity vs. detail, avoiding bias in summaries

Input data

- Articles from local folders.
- Type of files: pdf, doc, msg, pptx, etc.

Prompt Design

```
chat <- chat_azure_core(
  system_prompt = "You are a senior data analyst"
)

type_summary <- type_object(
  title=type_string("title of the article"),
  FirstNM=type_string("First Name of the article's first author"),
  LastNM=type_string("Last Name of the article's first author"),
  Pubdate=type_string("Publication date as formart month/day/year with two numbers in
month and day, 4 numbers in year"),
  Journal = type_string("journal of the article"),
  DOI = type_string("doi of the article"),
  Summary=type_string("Summary of the article, 50 words"),
  Methods=type_string("list the names of the statistical methods used in this article"),
  Population=type_string("population of this article"))
```

Output Table

| Title | FirstNM | LastNM | Pubdate | Journal | DOI | Summary | Methods | Population |
|--|---------|-----------|-----------|--------------------|----------------------------|--|--|--|
| Pregnant women and substance use: fear, stigma, and barriers to care | Rebecca | Stone | 3/15/2015 | Health and Justice | 10.1186/s40352-015-0015-5 | This study explores the experiences of substance-using pregnant women who face fear, stigma, and barriers to accessing care. Women practice avoidance strategies to prevent detection and involvement with legal authorities. Findings emphasize the need for policy changes to support comprehensive treatment options that consider women's nuances and challenges. | Qualitative analysis, in-depth life history interviews | 30 recently-pregnant women |
| Integrating Social Services & Health: Project Nurture integrates Care and Services to Improve Outcomes For Opioid-Dependent Mothers And Their Children | K. John | McConnell | 4/1/2020 | Health Affairs | 10.1377/hlthaff.2019.01574 | Project Nurture in Portland, Oregon, integrates maternity care, substance use treatment, and social services for opioid-dependent mothers. It resulted in reductions in child maltreatment and foster care placements and increased prenatal visits and maternal hospital stays. The project suggests healthcare engaging human services can improve outcomes despite sustainability challenges. | difference-in-differences analysis | opiod-dependent women enrolled in Medicaid in Portland, Oregon |

Considerations

- Data security and privacy
- Cost and time
- Token limitation
- Static knowledge
- Bias reproduction
- Ethical concerns about responsibility
- Input data copy right issues

Suggestions for your adoption

- Prompt design
- Code Optimization
- Keep LLM model version updated
- Develop validation methods for your use cases
- Establishing lines of accountability



Key Takeaways



- LLMs can help with the qualitative analysis
- {ellmer} provides a simple way to implement LLMs
- Three practical applications with prompt design
- Limitation and suggestions in LLM applications.

Acknowledgements

THANK YOU TO EVERYONE WHO MADE THIS WORK POSSIBLE!

- Daniel J. Fish, PhD
- Megan Holtorf, MPH, CCRP
- Yan Liu, MS, MM



Resources

- {ellmer}: implement LLM from R [Chat with Large Language Models • ellmer](#)
- {hellmer}: batch processing for chat models [CRAN: Package hellmer](#)
- {rtiktoken}: get tokens for Open AI models [CRAN: Package rtiktoken](#)
- {tidyllm}: A tidy interface for integrating large language model (LLM) APIs such as 'Claude', 'Openai', 'Gemini', 'Mistral' and local models via 'Ollama' into R workflows. <https://cran.r-project.org/web/packages/tidyllm/index.html>
- {tidyprompt}: Easily construct prompts and associated logic for interacting with large language models (LLMs) . <https://cran.r-project.org/web/packages/tidyprompt/index.html>
- <https://github.com/jcheng5/rmedicine-2025> : Demystifying LLMs with Ellmer
- [Whitepaper on Prompt Engineering by Google : Google : Free Download, Borrow, and Streaming : Internet Archive](#)
- <https://www.promptingguide.ai/introduction/tips>
- <https://archive.org/details/whitepaper-prompt-engineering-v-4/page/7/mode/1up>
- <https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/>



Scan bar code to get repository for more information!